



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2
Розробка мобільних застосунків під Android
«ДОСЛІДЖЕННЯ РОБОТИ З КОМПОНЕНТОМ FRAGMENT»
Варіант 8

Виконав:

студент групи ІА-31

Дементій О. Є.

Київ 2026

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набуті практичні навички з використання фрагментів для інтерфейсу користувача.

Завдання на лабораторну роботу

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («ОК»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel», яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності. Примітка: завдання відповідає варіанту лабораторної роботи №1.

Хід роботи

1. Створюємо проект

2. В класі MainActivity пишемо такий код:

```
package com.example.lab2;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Якщо додаток тільки запустився, завантажуюмо перший фрагмент
        if (savedInstanceState == null) {
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.fragment_container, new InputFragment())
                .commit();
        }
    }
}
```

3. В класі InputFragment пишемо такий код:

```
package com.example.lab2;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import android.widget.RadioGroup;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;

public class InputFragment extends Fragment {

    private CalculatorViewModel viewModel;

    @Override
    public View onCreateView(LayoutInflater inflater,
        ViewGroup container, Bundle savedInstanceState) {
        return
inflater.inflate(R.layout.fragment_input, container,
false);
    }

    @Override
    public void onViewCreated(@NonNull View view,
        @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        // Підключаємо спільну ViewModel
        viewModel = new
ViewModelProvider(requireActivity()).get(CalculatorView
Model.class);

        // Знаходимо всі елементи інтерфейсу
        EditText etNum1 = view.findViewById(R.id.num1);
        EditText etNum2 = view.findViewById(R.id.num2);
        RadioGroup opGroup =
view.findViewById(R.id.opGroup);
        Button btnOk = view.findViewById(R.id.btnOk);

        btnOk.setOnClickListener(v -> {
            try {
                // Зчитуємо дані
```

```

        double n1 =
Double.parseDouble(etNum1.getText().toString());
        double n2 =
Double.parseDouble(etNum2.getText().toString());
        double res = 0;
        int selectedId =
opGroup.getCheckedRadioButtonId();

        // Виконуємо операцію за варіантом 8
        if (selectedId == R.id.rbPlus) {
            res = n1 + n2;
        } else if (selectedId == R.id.rbMinus)
{
            res = n1 - n2;
        } else if (selectedId == R.id.rbMult) {
            res = n1 * n2;
        } else if (selectedId == R.id.rbDiv) {
            if (n2 == 0) {
                viewModel.setResult("Помилка:
ділення на 0");

                navigateToResult();
                return;
            }
            res = n1 / n2;
        } else {
            viewModel.setResult("Виберіть
операцію");

            navigateToResult();
            return;
        }

        // Зберігаємо результат і переходимо
viewModel.setResult("Результат: " +
res);

        navigateToResult();

    } catch (NumberFormatException e) {
        viewModel.setResult("Введіть коректні
числа");

        navigateToResult();
    }
    });
}

private void navigateToResult() {
    getParentFragmentManager().beginTransaction()
        .replace(R.id.fragment_container, new
ResultFragment())

```

```

        .addToBackStack (null)
        .commit ();
    }
}

```

4. В класі ResultFragment пишемо такий код:

```

package com.example.lab2;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.lifecycle.ViewModelProvider;

public class ResultFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_result, container, false);
    }

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        CalculatorViewModel viewModel = new
ViewModelProvider(requireActivity()).get(CalculatorViewModel.class);

        TextView tvResult = view.findViewById(R.id.tvResult);
        Button btnCancel = view.findViewById(R.id.btnCancel);

        // Спостерігаємо за результатом у ViewModel
        viewModel.getResult().observe(getViewLifecycleOwner(), result -> {
            if (result != null) {
                tvResult.setText(result);
            }
        });

        // Кнопка Cancel повертає назад і очищає дані за завданням
        btnCancel.setOnClickListener(v -> {

```

```

        viewModel.clear(); // Метод clear ми додали у ViewModel
        getParentFragmentManager().popBackStack();
    });
}
}

```

5. Файл розмітки activity_main.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
</FrameLayout>

```

6. Файл ресурсів strings.xml:

```

<resources>
    <string name="app_name">Lab2</string>
    <!-- TODO: Remove or change this placeholder text -->
    <string name="hello_blank_fragment">Hello blank
fragment</string>
</resources>

```

7. Файл маніфесту AndroidManifest.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Lab2">
        <activity
            android:name=".MainActivity"

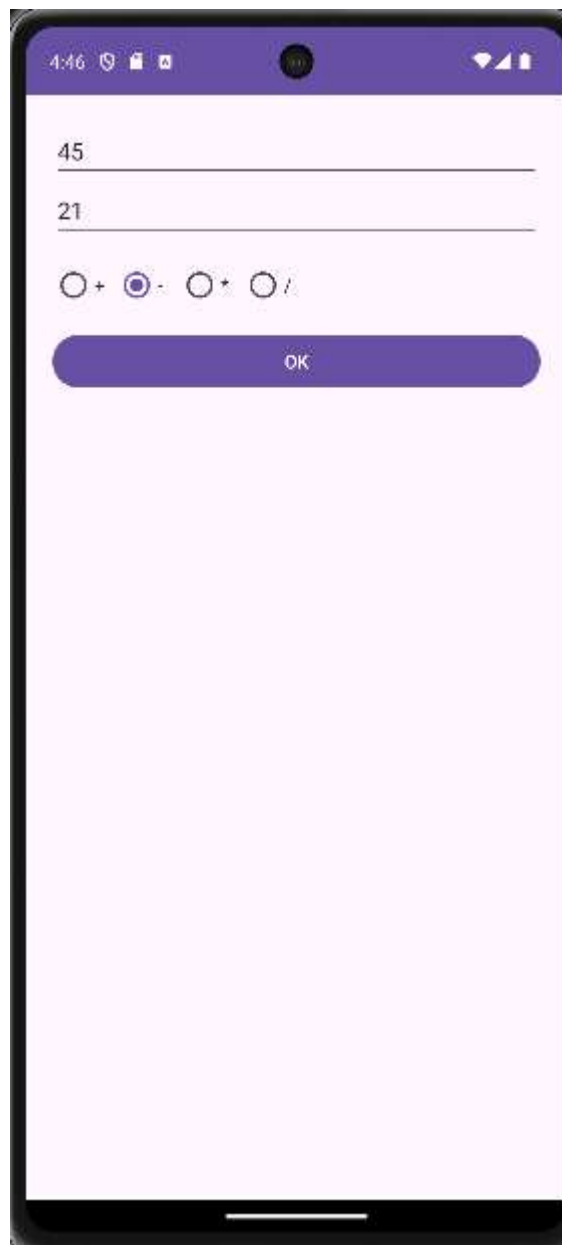
```

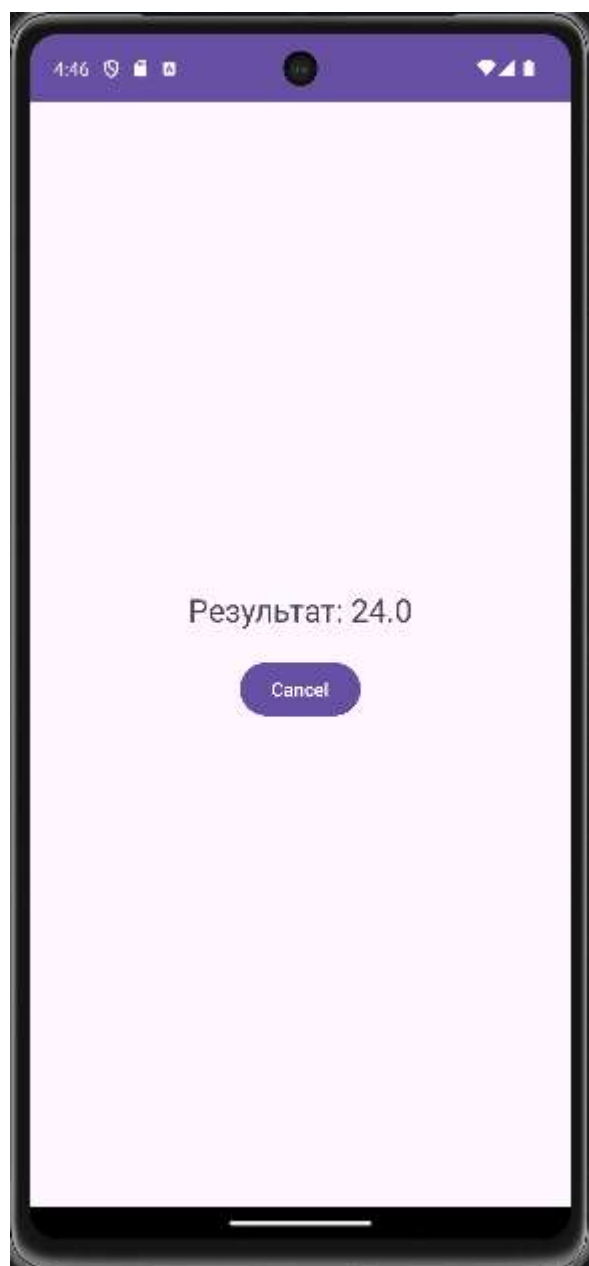
```
        android:exported="true">
        <intent-filter>
            <action
android:name="android.intent.action.MAIN" />

            <category
android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>

</manifest>
```

Результат:





Висновок: У ході виконання лабораторної роботи №2 було досліджено механізми створення та взаємодії компонентів **Fragment** в операційній системі Android. На практиці реалізовано модульну архітектуру застосунку, де інтерфейс розділено на логічні блоки (введення та виведення результату).

Опановано використання **Shared ViewModel** для забезпечення надійного обміну даними між фрагментами та збереження стану при зміні конфігурації пристрою. Набуто навичок роботи з `FragmentManager` для динамічної заміни екранів та керування стеком переходів, що дозволило створити зручний та сучасний інтерфейс користувача згідно з обраним варіантом.

.